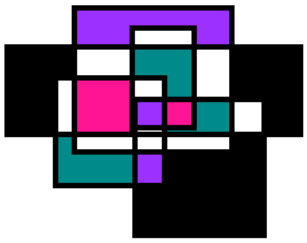
Disciplina Compiladores - UFS
Turmas T01 e T02 2026.2
Profa. Beatriz Trinchão Andrade



Especificação da Linguagem Quinas

1. Introdução

Já pensaram sobre como seriam as linguagens de programação se certas escolhas fossem feitas de forma diferente? Se, por exemplo, quando alguém especificou o que seria o primeiro comando condicional, ao invés de usar a palavra if, pensasse em outro símbolo? E se ele, e não o if, se tornasse o padrão? E se o português fosse a língua mais usada em programação? Faria sentido, pelo menos no Brasil.

Assim, indo contra alguns padrões vigentes na computação, a linguagem **Quinas** é uma linguagem funcional e em português que contém características – muito simplificadas e aleatórias - de ML, Scheme, LISP e Haskell. Ela também tem uma estética composta por linhas retas, de onde deriva o seu nome.

Quinas, obviamente, é uma linguagem experimental, então esta especificação é passível de adaptações. Em caso de modificações na especificação, versões atualizadas serão postadas via SIGAA (no tópico de aula "Definições do Projeto") e notificações serão enviadas às alunas e alunos.

As Seções 2, 3, 4 e 5 deste documento apresentam a especificação da linguagem. A Seção 6 contém informações sobre a avaliação e entregas. Serão anexados ao SIGAA exemplos de código fonte em **Quinas** (extensão .qui).

2. Características principais e léxico

Regras para identificadores:

- Identificadores podem conter duas ou mais letras, em qualquer caixa.
- Identificadores de função devem começar com uma letra maiúscula.
- Não são permitidos espaços em branco, acentos, underline ou caracteres especiais (ex.: _, á, @, $, ç, +, -, ^, % etc.).
- Identificadores não podem ser iguais às palavras reservadas ou operadores da linguagem.

Regras para identificadores:

- Identificadores podem conter duas ou mais letras.
- Não são permitidos espaços em branco, acentos, underline ou caracteres especiais (ex.: _, á, @, $, ç, +, -, ^, % etc.).
- Identificadores de constantes devem ter todas as letras maiúsculas.
- Identificadores de função devem começar com uma letra maiúscula (sugestão: usar estilo CamelCase).
- Identificadores de parâmetros de função que são de tipos primitivos devem estar em letras minúsculas.
- Identificadores não podem ser iguais às palavras reservadas ou operadores da linguagem.

Tipos primitivos:

- A linguagem aceita os tipos **numero** e **booleano**.
- Números inteiros podem ser escritos em decimal ou em binário (começando com 0b).
- A parte decimal dos números reais deve ser separada por uma vírgula.
- Booleanos podem assumir dois valores: V e F.

Tipos compostos:

- Não possui.

Blocos:

- Blocos são delimitados por colchetes.

Comentários:

- A linguagem aceita comentários de linha, indicados pelo símbolo -- no início da linha.
- A linguagem aceita comentários de bloco (possivelmente de múltiplas linhas) delimitados por /-- e --/
- O funcionamento dos comentários de bloco em **Quinas** são similares aos da linguagem C. Exemplo: o que acontece se você compilar /* */ */ em C? E isso: /* /* */? Estudem como um compilador C reconhece o fim de um comentário de bloco.

Estruturas de controle:

- repetição: recursão
- condição: guardas

Operadores:
Operadores aritméticos:

+	(soma)
-	(subtração ou menos unário)
.	(multiplicação)
/	(divisão real)
\	(resto)
//	(divisão inteira)
´	(operador de potência – caractere do acento agudo)

Operadores relacionais:

>>	(maior que)
<<	(menor que)
==	(igualdade)

- Operadores booleanos:

^	(e)
¬	(não unário – caractere no Alt Gr +6)
v	(ou)

- A precedência dos operadores é igual à de C, e pode ser alterada com o uso de < e >.

Subrotinas:

- Funções, sempre com retorno de valor

Vide Seção 3 para mais informações sobre a sintaxe dos comandos e sobre as classes de tokens a serem implementadas.

3. Sintático

A gramática da linguagem foi escrita seguindo as seguintes convenções:

- Variáveis da gramática são escritas em letras minúsculas sem aspas;
- Lexemas são escritos entre aspas simples;
- Símbolos escritos em letras maiúsculas representam o lexema de um token do tipo especificado;
- Os operadores da gramática seguem padrão similar ao visto em aula.

```
programa :   dec_funcao+

dec_funcao:'++'? '[' tipo IDFUNCAO '[' lista_parametros ']' ':' '[' exp ']' ']'

tipo : 'numero' | 'booleano'

lista_parametros :
        ε
      | parametro ( '|' parametro )*
parametro : tipo ID | assinatura
assinatura : tipo IDFUNCAO '[' parametros_assinatura ']'

//os parâmetros na assinatura de uma função passada como parametro não têm
//identificador
parametros_assinatura:
        ε
      | parametro_assinatura (|' parametro_assinatura)*

parametro_assinatura : tipo | assinatura

exp : NUMERAL_INT | NUMERAL_REAL | 'V' | 'F'
      | ID
      | IDCONST
      | bloco_exp
      | chamada
      | guarda
      | '<' exp '>'
      | '-' exp
      | exp ''' exp
      | exp '+' exp
      | exp '-' exp
      | exp '.' exp
      | exp '/' exp
      | exp '//' exp
      | exp '\' exp
      | exp '==' exp
      | exp '<<' exp
      | exp '>>' exp
      | '¬' exp
      | exp '^' exp
      | exp 'v' exp

guarda : '[' ('||' exp '=' exp)+ ('||' '#' '=' exp) ']'

bloco_exp : '[' dec_cons+ exp ']'
dec_cons : '[' '¨' tipo ID IDCONST '[' exp ']' ']'
chamada : IDFUNCAO '[' lista_args ']'

lista_args :
       ε
     | (arg)  ('|' arg )*

arg = exp
    | IDFUNCAO
```

4. Semântico:

- A execução de um programa consiste na execução de uma função com marcador ++. No máximo uma função deve ter essa marcação em um programa. Um programa sem esta marcação gera um erro de compilação.

- Em operações entre valores de tipos inteiro e real, os valores inteiros devem ser convertidos para real.

- A linguagem possui um conjunto de funções matemáticas predefinidas. É só usar essas funções no código, caso precisem. Assumiremos que elas são funções primitivas da linguagem, que podem ser usadas sem a necessidade de definição ou de importação de bibliotecas externas. São elas:

```
numero Seno[numero valor] --ângulo em radiano
numero Cosseno[numero valor] --ângulo em radiano
numero Tangente[numero valor] --ângulo em radiano
numero Logaritmo[numero valor] --base 10
```

- Funções de alta ordem em Quinas: uma função pode receber uma ou mais funções como parâmetros. Exemplo:

```
[ numero Max [numero xx | numero yy] :
       [
          || <xx >> yy> = xx
          || # = yy
        ]
]


[ numero AplicaOperador [ numero xx | numero yy | numero Op[ numero | numero] ] :
      [ Op[xx | yy] ] -- função de alta ordem
]


++ [ numero Principal [] :
       [ AplicaOperador[7 | 5 | Max] ]
```

O que deve ser verificado na análise semântica:

- Se as entidades criadas pelo usuário são inseridas na tabela de símbolos - com os atributos necessários - quando são declaradas;
- Se uma entidade foi declarada e está em um escopo válido no momento em que ela é utilizada;
- Se entidades foram declaradas e definidas (inicializadas) quando isso se fizer necessário;
- Checar a compatibilidade dos tipos de dados envolvidos nas declarações, vínculos, chamadas de função e expressões.

5. Geração de Código

- O código alvo do compilador pode ser Haskell ou C.
- As entradas/saídas dos programas executáveis são definidas pela função que contém o marcador de ponto de entrada, e podem ser obtidas/exibidas nesta etapa como comandos de leitura e escrita em console.

6. Desenvolvimento do Trabalho

Trabalhos devem ser desenvolvidos em quartetos (preferencialmente), trios, duplas ou individualmente (mediante justificativa). Os grupos devem se cadastrar na planilha correspondente à turma dos seus componentes:

https://docs.google.com/spreadsheets/d/1KkcVQwKO19lxLCyxvmHEyz8myXxfQSjJOYMJLs6h8xI/edit?usp=sharing

Prazo para preenchimento da planilha: 11/09/2026

Serão abertos fóruns no SIGAA para a discussão sobre as etapas. Em caso de dúvida, verifique inicialmente no fórum se ela já foi resolvida. Se ela persiste, consulte os monitores ou a professora.

6.1. Ferramentas

- Implementação com SableCC, linguagem Java.
- IDE Java (recomendação: Eclipse).
- Submissão das etapas do projeto via SIGAA. Haverá uma tarefa para cada etapa.

6.2. Avaliação

- A avaliação será feita com base nas etapas entregues e em entrevistas feitas com os grupos durante as aulas de acompanhamento do projeto.
- O valor de cada etapa está definido no plano de curso da disciplina.
- O cumprimento das requisições de formato também será avaliado na nota de cada etapa.

6.3. Etapas

Etapa 1. Códigos Quinas
- **Prazo: 17/09/2026**
- **Atividade:** escrever três códigos em **Quinas** que, unidos, usem todas as alternativas gramaticais (ou seja, todos os recursos) da linguagem.
- **Formato de entrega:** arquivo comprimido contendo três códigos, onde cada código deve estar escrito em um arquivo de texto simples, com extensão "**.qui**".

Etapa 2. Análise Léxica
- **Prazo: 01/10/2026**
- **Atividade:** implementar analisador léxico da linguagem com o SableCC, fazendo a impressão dos lexemas e tokens reconhecidos ou imprimindo erro quando o token não for reconhecido.
- **Formato de entrega:** apenas o arquivo .sable deve ser enviado. O nome do arquivo deve ser `grupo_X.sable`, onde `X` é o número do grupo (vide planilha de cadastro de grupos).
  O nome do pacote a ser gerado pelo sablecc deve se chamar **quinas** (em letras minúsculas).

Etapa 3. Análise Sintática
- **Prazo: 05/11/2026**
- **Atividade:** implementar analisador sintático da linguagem com o SableCC, fazendo impressão da árvore sintática em caso de sucesso ou impressão dos erros caso contrário.

- **Formato de entrega:** apenas o arquivo .sable deve ser enviado. O nome do arquivo deve ser `grupo_X.sable`, onde `X` é o número do grupo (vide planilha de cadastro de grupos).
  O nome do pacote a ser gerado pelo sablecc deve se chamar **quinas** (em letras minúsculas).

Etapa 4. Análise Sintática Abstrata

- **Prazo: 24/11/2026**
- **Atividade:** implementar analisador sintático abstrato em SableCC, com impressão da árvore sintática.
- **Formato de entrega:** apenas o arquivo .sable deve ser enviado. O nome do arquivo deve ser `grupo_X.sable`, onde `X` é o número do grupo (vide planilha de cadastro de grupos).
  O nome do pacote a ser gerado pelo sablecc deve se chamar **quinas** (em letras minúsculas).

Etapa 5. Tabela de Símbolos e Análise Semântica

- **Prazo: 03/12/2026**
- **Atividade:** implementação da tabela de símbolos, validar escopo, declaração e definição de identificadores. Implementar verificação de tipos (incluindo polimorfismo e herança).
- **Formato de entrega:** projeto completo, incluindo obrigatoriamente: o arquivo .sable; todas as classes java escritas pelo grupo ou geradas automaticamente; e arquivos .**qui** que demonstrem o que foi feito nesta tarefa.
  Também é obrigatória a entrega de um pdf contendo uma breve explicação sobre o que foi implementado nesta etapa e como.

Etapa 6. Geração de código (extra: 2 pontos na segunda unidade):

- **Prazo: o mesmo da Etapa 5**
- Compilação de código **Quinas** com geração de código em linguagem alvo.

Observações importantes:

- Entregas após o prazo sofrem penalidade de metade da nota da etapa por dia de atraso.
- Trabalhos entregues com atraso devem ser submetidos na Tarefa ‘Entrega após prazo’, no SIGAA, que ficará aberta durante todo o período.
- Arquivos enviados por e-mail não serão considerados.

Bom trabalho!